ALLIANCE UNIVERSITY | NAAC GRADE A+ ACCREDITED UNIVERSITY

Database Management System

Assignment 3

CAB BOOKING SYSTEM

Name	Razin Basha R
Roll No	2511021240012
Course	Database Management System
Programme	BCA AI/ML
Semester	III

Introduction

A cab booking system is a database-based system that lets customers book a cab to travel from one place to another. In this assignment I have chosen the Cab Booking System as my domain. The system keeps and manages important information such as customer details, driver details, car details, bookings and payments.

Everything is stored in a database so that the data stays organised and is easy to find whenever it is needed. I have used primary keys to identify every record uniquely and foreign keys to connect one table with another. Overall, this system is useful for managing cab bookings and all the information related to them.

Name and Purpose of the Application

Application name: Cab Booking System

Purpose: The Cab Booking System is a database management system where customers can reserve a cab for their travel. It stores the data about customers, drivers, cars, bookings and payments. With this system we can easily find out which customer booked which car, which driver is going to take them, and how much they paid for the trip.

Tables Required

To build this system I need five tables:

- Customer
- Driver
- Car
- Booking
- Payment

Entities and Attributes

The system has five main entities. These are the same five I used in my first DBMS assignment. The primary key (PK) of each entity is marked in the table below.

Entity	Primary Key	Other Attributes
CUSTOMER	Customer_ID (PK)	Customer_Name, Phone, Email, Address
DRIVER	Driver_ID (PK)	Driver_Name, Phone, License_No, Availability
CAR	Car_ID (PK)	Car_Number, Car_Model, Car_Type, Fare_Per_KM
BOOKING	Booking_ID (PK)	Booking_Date, Pickup_Location, Drop_Location, Distance_KM, Booking_Status
PAYMENT	Payment_ID (PK)	Amount, Payment_Date, Payment_Method, Payment_Status

Columns of Each Table

Customer Table

Column	Description	Key
Customer_ID	Unique ID of the customer	Primary Key
Customer_Name	Name of the customer	-
Phone	Phone number of the customer	-
Email	Email of the customer	-
Address	Address of the customer	-

Driver Table

Column	Description	Key
Driver_ID	Unique ID of the driver	Primary Key
Driver_Name	Name of the driver	-
Phone	Phone number of the driver	-
License_No	Driving license number	-
Availability	Whether the driver is available or not	-

Car Table

Column	Description	Key
Car_ID	Unique ID of the car	Primary Key
Car_Number	Vehicle registration number	-
Car_Model	Model of the car	-
Car_Type	Sedan, SUV, Hatchback, etc.	-
Fare_Per_KM	Cost per kilometre	-
Driver_ID	Driver assigned to the car	Foreign Key

Booking Table

Column	Description	Key
Booking_ID	Unique ID of the booking	Primary Key
Customer_ID	Customer who made the booking	Foreign Key
Car_ID	Car that was booked	Foreign Key
Booking_Date	Date of the booking	-
Pickup_Location	Starting location	-
Drop_Location	Destination	-
Distance_KM	Travel distance in km	-
Booking_Status	Status of the booking	-

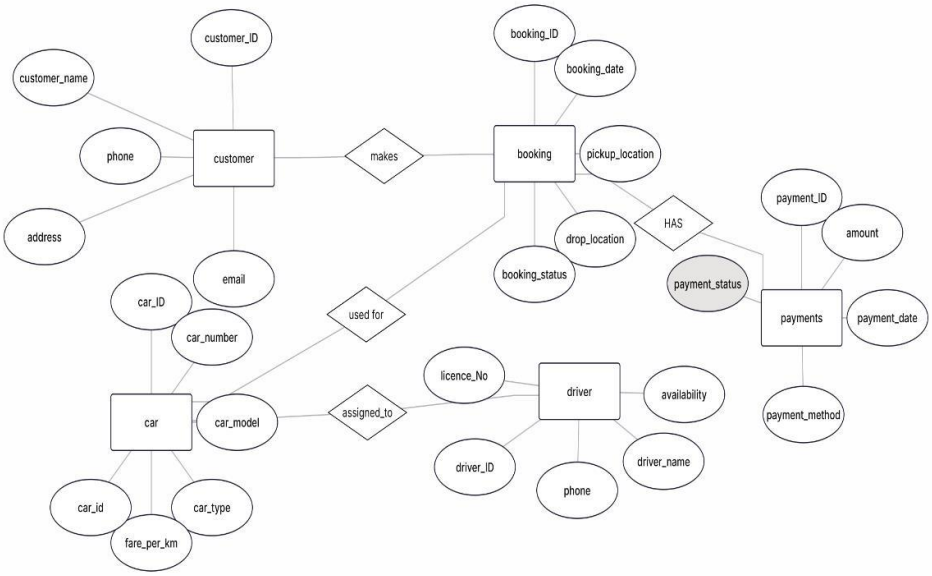
Payment Table

Column	Description	Key
Payment_ID	Unique ID of the payment	Primary Key
Booking_ID	Booking linked with the payment	Foreign Key
Amount	Amount paid	-
Payment_Date	Date of the payment	-
Payment_Method	Cash / UPI / Card	-
Payment_Status	Paid / Pending	-

Relationships and Cardinality

Relationship	Cardinality	Description
CUSTOMER makes BOOKING	One-to-Many (1:N)	A customer can make many bookings over time, but each booking is made by exactly one customer.
DRIVER is assigned to CAR	One-to-Many (1:N)	A driver can be assigned to many cars, but each car is assigned to exactly one driver at a time.
CAR is used for BOOKING	One-to-Many (1:N)	A car can be used for many bookings over time, but each booking uses exactly one car.
BOOKING has PAYMENT	One-to-One (1:1)	Each booking generates exactly one payment record, and each payment is linked to exactly one booking.

ER Diagram



Relational Schema

Next I converted the ER diagram into a relational schema. Each entity becomes one table. For every 1:N relationship, the foreign key is placed on the "many" side, and for the 1:1 relationship it can be placed on either side. My design has no many-to-many relationships, so I did not need any extra junction tables.

CUSTOMER

- Customer_ID (PK)
- Customer_Name
- Phone
- Email
- Address

DRIVER

- Driver_ID (PK)
- Driver_Name
- Phone
- License_No

- Availability

CAR

- Car_ID (PK)
- Car_Number
- Car_Model
- Car_Type
- Fare_Per_KM
- Driver_ID (FK → DRIVER)

BOOKING

- Booking_ID (PK)
- Customer_ID (FK → CUSTOMER)
- Car_ID (FK → CAR)
- Booking_Date
- Pickup_Location
- Drop_Location
- Distance_KM
- Booking_Status

PAYMENT

- Payment_ID (PK)
- Booking_ID (FK → BOOKING)
- Amount
- Payment_Date
- Payment_Method
- Payment_Status

Relationship Explanation

CUSTOMER – MAKES – BOOKING (1:N)

One customer can make many bookings over time, for example different trips on different dates. But every single booking is made by only one customer. To show this, I stored Customer_ID as a foreign key in the BOOKING table.

DRIVER – ASSIGNED TO – CAR (1:N)

A driver can be assigned to more than one car over time, but at any given moment a car has only one driver. So Driver_ID is placed as a foreign key in the CAR table.

CAR – USED FOR – BOOKING (1:N)

A single car can be used for many bookings during its lifetime, but each booking uses just one car for that trip. This is shown by keeping Car_ID as a foreign key in the BOOKING table.

BOOKING – HAS – PAYMENT (1:1)

Every booking results in one payment record, and every payment belongs to one booking only. I implemented this by storing Booking_ID as a foreign key in the PAYMENT table.

10. Sample Data

Here is some example data for each table to show how the records look.

Customer Table

Customer_ID	Customer_Name	Phone	Email	Address
CU01	Rahul	9876543210	rahul@mail.com	Bangalore
CU02	Arjun	9876543211	arjun@mail.com	Chennai
CU03	Kiran	9876543212	kiran@mail.com	Krishnagiri

Driver Table

Driver_ID	Driver_Name	Phone	License_No	Availability
D01	Ravi	9123456780	DL1001	Available
D02	Kumar	9123456781	DL1002	Busy
D03	Suresh	9123456782	DL1003	Available

Car Table

Car_ID	Car_Number	Car_Model	Car_Type	Fare_Per_KM	Driver_ID
C01	KA01AB1234	Swift	Hatchback	₹12	D01
C02	KA02CD5678	City	Sedan	₹15	D02
C03	KA03EF9012	Creta	SUV	₹20	D03

Booking Table

Booking_ID	Customer_ID	Car_ID	Booking_Date	Pickup	Drop	Distance	Status
B01	CU01	C01	07-09-2026	Airport	MG Road	25 km	Completed
B02	CU02	C02	07-09-2026	Mall	Railway Station	12 km	Completed
B03	CU03	C03	08-09-2026	College	Airport	30 km	Confirmed

Payment Table

Payment_ID	Booking_ID	Amount	Payment_Date	Payment_Method	Status
P01	B01	₹300	07-09-2026	UPI	Paid

Payment_ID	Booking_ID	Amount	Payment_Date	Payment_Method	Status
P02	B02	₹180	07-09-2026	Card	Paid
P03	B03	₹600	08-09-2026	UPI	Pending

Purpose of Each Table

No.	Table	Purpose
1	Customer	Stores the details of customers such as customer ID, name, phone number, email and address.
2	Driver	Stores information about drivers, including driver ID, name, phone number, license number and availability.
3	Car	Stores details of the cars such as car ID, registration number, model, type, fare per kilometre and the assigned driver.
4	Booking	Stores information about each booking, including the customer, the car, booking date, pickup location, drop location, distance and booking status.
5	Payment	Stores the payment details of bookings such as payment ID, booking ID, amount, payment date, payment method and payment status.

Conclusion

The Cab Booking System is a good way to manage customers, drivers, cars, bookings and payments in one place. By using related tables, all the data is kept in a clean and well-organised manner. The primary key identifies each record in a table uniquely, while the foreign key connects one table to another. Working on this assignment helped me understand how an ER diagram is converted into a relational schema and how the tables in a database are linked together.